

MOwNiT - laboratorium 1

Analiza Błędów

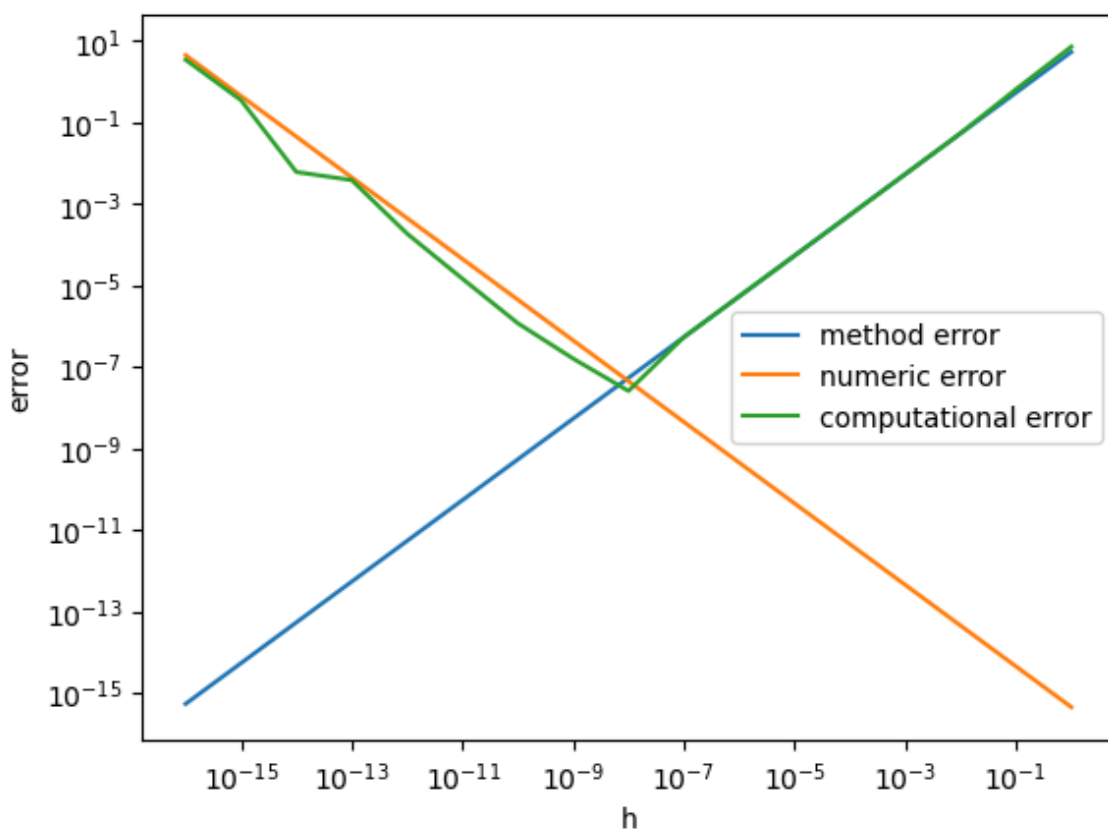
Hubert Kukła, Maksymilian Siemek - 11.03.2026

Zadanie 1

Sprawdzenie dokładności przybliżania pochodnej za pomocą wzoru $f'(x) = \frac{f(x+h)-f(x)}{h}$ oraz $f'(x) = \frac{f(x+h)-f(x-h)}{2h}$ dla funkcji $f(x) = \tan(x)$, $x = 1$. Porównanie dokładności obu metod, obliczenie błędu porównując je do pochodnej obliczonej analitycznie. Sprawdzenie działania metod dla $h = 10^{\{-k\}}$, gdzie $k = 1, 2, \dots, 16$. Obliczenie oraz wyznaczenie empiryczne wartości $h_{\min}$.

Różnica prawostronna (wzór pierwszy):

Błąd dla różnicy prawostronnej: $E(h) \leq \frac{Mh}{2} + \frac{2\varepsilon}{h} h_{\min} = 2\sqrt{\frac{\varepsilon_{\text{mach}}}{M}}$ gdzie $M \approx |f''(x)|$.



Rysunek 1: Wartości bezwzględne błędów metody, numerycznego i obliczeniowego

Minimum funkcji błędu obliczeniowego $E(h)$ wyznaczone empirycznie to $h_{\min} = 10^{\{-8\}}$.

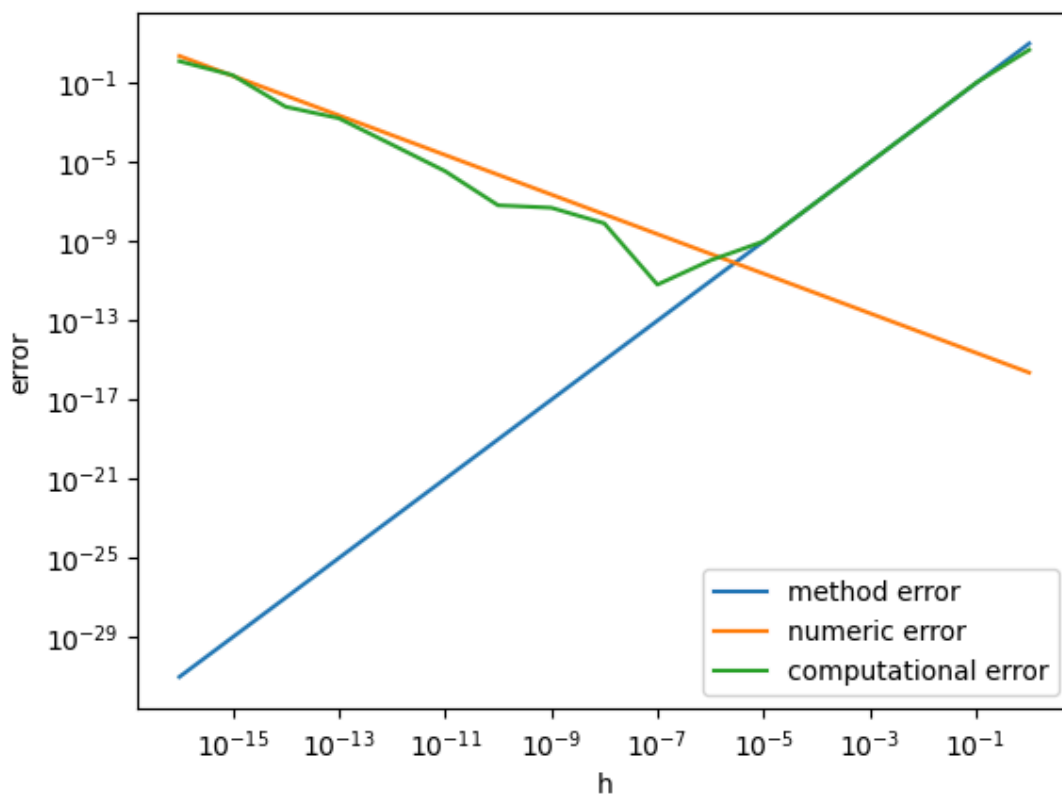
Wyznaczone analitycznie to $h_{\min} \approx 9.124 \cdot 10^{\{-9\}}$.

Różnica centralna (wzór drugi):

Błąd dla różnicy centralnej: $E(h) \leq \frac{Mh^2}{6} + \frac{\varepsilon}{h}$

$$h_{\min} = \sqrt[3]{3 \frac{\varepsilon_{\text{mach}}}{M}}$$

, gdzie $M \approx |f'''(x)|$.



Rysunek 2: Wartości bezwzględne błędów metody, numerycznego i obliczeniowego

Minimum funkcji błędu obliczeniowego $E(h)$ wyznaczone empirycznie to $h_{\min} = 10^{-7}$.

Wyznaczone analitycznie to $h_{\min} \approx 2.273 \cdot 10^{-12}$.

Porównanie metod

- Minimalny błąd różnicy prawostronnej: $E(h_{\min}) \approx 2.554 \cdot 10^{-8}$
- Minimalny błąd różnicy centralnej: $E(h_{\min}) \approx 6.223 \cdot 10^{-12}$

Metoda różnicy centralnej ma mniejszy minimalny błąd, więc jest dokładniejsza.

Zadanie 2

Przepisywanie wyrażeń w celu zmniejszenia błędu numerycznego dla wskazanych argumentów:

a) $\sqrt{x+1} - 1$, dla $x \approx 0$

Kiedy $x \approx 0$, $\sqrt{x+1} \approx 1$, co prowadzi do odejmowania bliskich sobie liczb (błąd kancelacji).

Mnożymy licznik i mianownik przez sprzężenie:

$$\sqrt{x+1} - 1 = (\sqrt{x+1} - 1) \frac{\sqrt{x+1} + 1}{\sqrt{x+1} + 1} = \frac{x+1-1}{\sqrt{x+1}+1} = \frac{x}{\sqrt{x+1}+1}$$

b) $x^2 - y^2$, dla $x \approx y$

Aby uniknąć błędu potęgowania zaokrąglonych liczb przed operacją odejmowania, stosujemy wzór skróconego mnożenia (odejmowanie $x - y$ wykonane jako pierwsze jest tu dokładne z mocy lematu Sterbenza):

$$x^2 - y^2 = (x - y)(x + y)$$

c) $\frac{1-\cos x}{\sin} x$, dla $x \approx 0$

W tym przypadku zarówno licznik, jak i mianownik dążą do zera, a w liczniku zachodzi błąd kancelacji. Mnożymy licznik i mianownik przez $(1 + \cos x)$ lub stosujemy tożsamości dla połowy kąta:

$$\frac{1-\cos x}{\sin} x = \frac{(1-\cos x)(1+\cos x)}{\sin x(1+\cos x)} = \frac{1-\cos^2 x}{\sin x(1+\cos x)} = \frac{\sin^2 x}{\sin x(1+\cos x)} = \frac{\sin x}{1+\cos x}$$

Opcjonalnie (z użyciem tangensa połowy kąta):

$$\frac{2 \sin^2(\frac{x}{2})}{2 \sin(\frac{x}{2}) \cos(\frac{x}{2})} = \tan\left(\frac{x}{2}\right)$$

d) $\sin x - \sin y$, dla $x \approx y$

Aby ominąć problem odejmowania dwóch bardzo podobnych wartości, korzystamy ze wzoru na różnicę sinusów, który zamienia odejmowanie na mnożenie:

$$\sin x - \sin y = 2 \sin\left(\frac{x-y}{2}\right) \cos\left(\frac{x+y}{2}\right)$$

e) $\frac{a+b}{2}$ (bisekcja odcinka $[a, b]$)

Dodanie do siebie dwóch bardzo dużych liczb tego samego znaku może spowodować przepełnienie (overflow). Bezpieczniej jest dodać do mniejszej z nich połowę ich różnicy:

$$\frac{a+b}{2} = a + \frac{b-a}{2}$$

f) $\text{softmax}(x)_i = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$ dla $x = [x_1, \dots, x_n]$

Dla dużych argumentów funkcja eksponencjalna szybko dąży do nieskończoności (overflow). Aby temu zapobiec, stabilizujemy wzór odejmując od każdego x element maksymalny z wektora $m =$

$\max(x)$. Wyrażenie nie zmienia swojej wartości matematycznej, ale zyskuje stabilność numeryczną (najwyższą potęgą będzie wynosić 0):

$$\text{softmax}(x)_i = \frac{\exp(x_i - m) \exp(m)}{\sum_j \exp(x_j - m) \exp(m)} = \frac{\exp(x_i - m)}{\sum_j \exp(x_j - m)}$$

Empiryczne porównanie wersji oryginalnej i stabilnej

Podpunkt a) $\sqrt{x+1} - 1$

Dla bardzo małego argumentu dodanie go do jedynki, a następnie spierwiastkowanie, powoduje obcięcie cyfr znaczących. W rezultacie odejmujemy od siebie bliskie jedynce liczby, co prowadzi do drastycznego błędu. Wersja stabilna omija ten problem.

- **Argument:** $x = 10^{\{-16\}}$ (`Decimal('1e-16')`)
- **Precyzja:** 16 (`getcontext().prec = 16`)
- **Wynik wzoru oryginalnego:** 0E-8 (całkowita utrata dokładności)
- **Wynik wzoru stabilnego:** 5E-17 (poprawne przybliżenie)

```
from decimal import Decimal, getcontext

getcontext().prec = 16
x = Decimal('1e-16')

original = (1 + x).sqrt() - 1
stable = x / ((1 + x).sqrt() + 1)

print("original:", original) # wynik: 0E-8
print("stable:", stable)     # wynik: 5E-17
```

Podpunkt c) $\frac{1-\cos x}{\sin x} x$

Dla bardzo małego kąta, standardowa funkcja cosinus zaokrągla wynik do dokładnie 1.0. Przez to licznik ułamka zrównuje się z zerem, a całe wyrażenie zwraca 0.0. Użycie tożsamości z tangensem z wersji stabilnej pozwala na bezbłędne policzenie wyniku.

- **Argument:** $x = 10^{\{-8\}}$ (`1e-8`)
- **Precyzja:** 16 (standardowa dla biblioteki math w Pythonie)
- **Wynik wzoru oryginalnego:** 0.0 (wyzerowanie licznika)
- **Wynik wzoru stabilnego:** 5e-09 (poprawne przybliżenie)

```
from decimal import getcontext
import math

getcontext().prec = 16
x = 1e-8

original = (1 - math.cos(x)) / math.sin(x)
stable = math.tan(x/2)

print("original:", original) # Wynik: 0.0
print("stable:", stable)     # Wynik: 5e-09
```

Zadanie 3

Rozważamy funkcję daną wzorem:

$$f(x) = \begin{cases} 1 & \text{dla } x = 0 \\ \frac{e^x - 1}{x} & \text{dla } x \neq 0 \end{cases}$$

1. Wykazanie granicy funkcji

Należy wykazać, że $\lim_{x \rightarrow 0} f(x) = 1$.

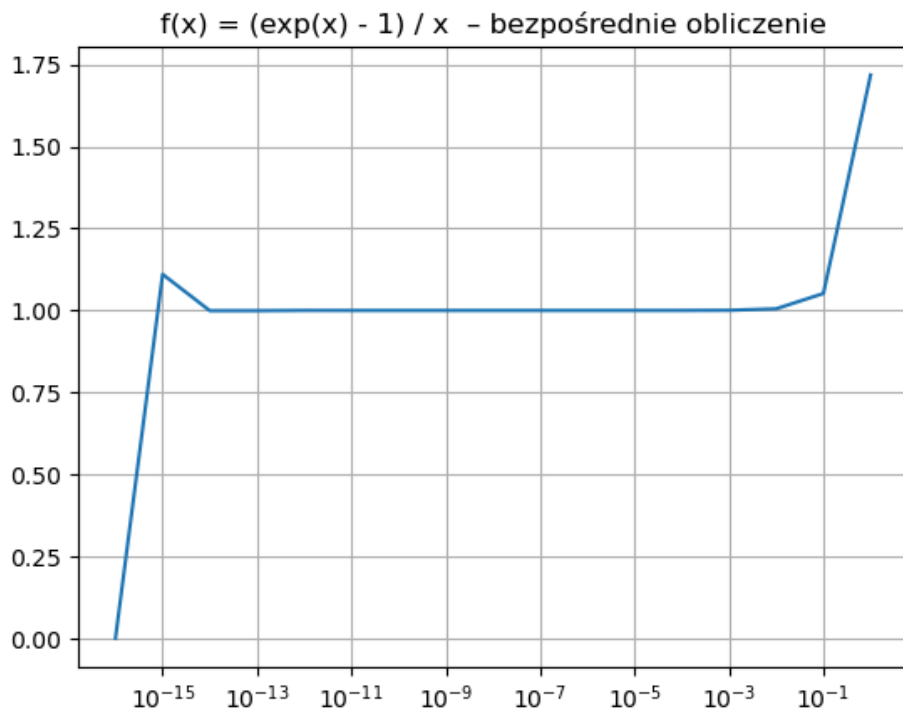
Ponieważ dla $x \rightarrow 0$ wyrażenie $\frac{e^x - 1}{x}$ daje symbol nieoznaczony $\frac{0}{0}$, możemy skorzystać z reguły de l'Hospitala, obliczając pochodne licznika i mianownika:

$$\lim_{x \rightarrow 0} \frac{e^x - 1}{x} = \lim_{x \rightarrow 0} \frac{\frac{\partial}{\partial x}(e^x - 1)}{\frac{\partial}{\partial x}x} = \lim_{x \rightarrow 0} \frac{e^x}{1} = e^0 = 1$$

Zatem pokazano, że $\lim_{x \rightarrow 0} f(x) = 1$.

2. Weryfikacja i wykres dla małych wartości x

W celu weryfikacji sporządzono wykres funkcji $f(x)$ dla $x = 10^{\{-k\}}$, gdzie $k = 0, \dots, 16$.



Rysunek 3: Wykres funkcji $f(x) = \frac{e^x - 1}{x}$ w zależności od x w skali logarytmicznej (dla $x = 10^{\{-k\}}$).

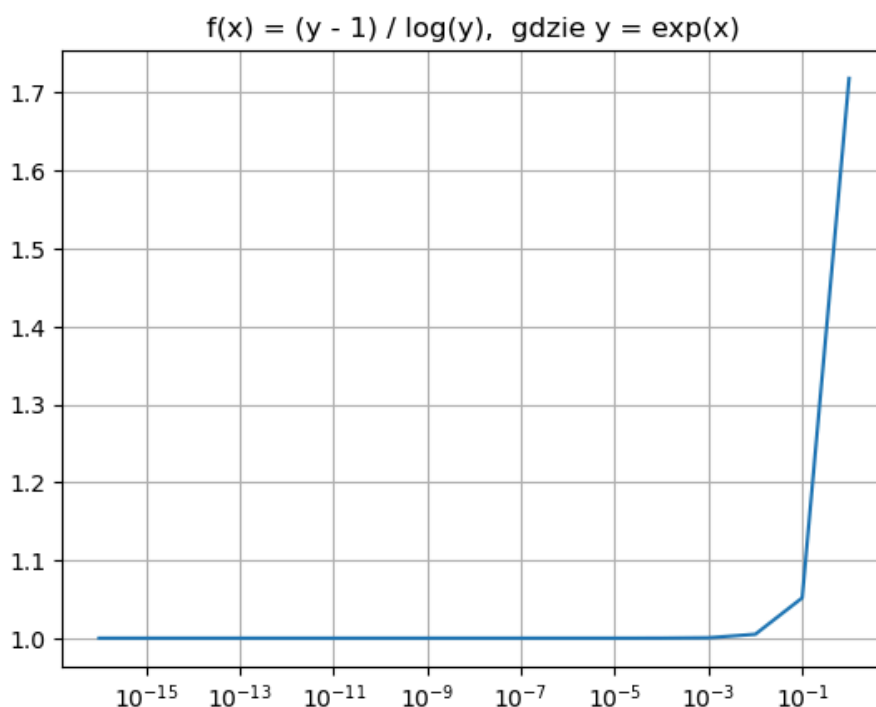
Jak widać, dla małych wartości x funkcja stabilizuje się na wartości 1, co jest zgodne z policzoną wcześniej granicą.

3. Równoważna formuła matematyczna

Zastosowano matematycznie równoważną formułę, wprowadzając pomocniczą zmienną y :

$$y = e^x$$

$$f(x) = \begin{cases} 1 & \text{dla } y = 1 \\ \frac{y-1}{\log(y)} & \text{dla } y \neq 1 \end{cases}$$



Rysunek 4: Wykres funkcji obliczonej z wykorzystaniem równoważnego wzoru z logarytmem naturalnym.

4. Aproxymacja za pomocą szeregu Taylora

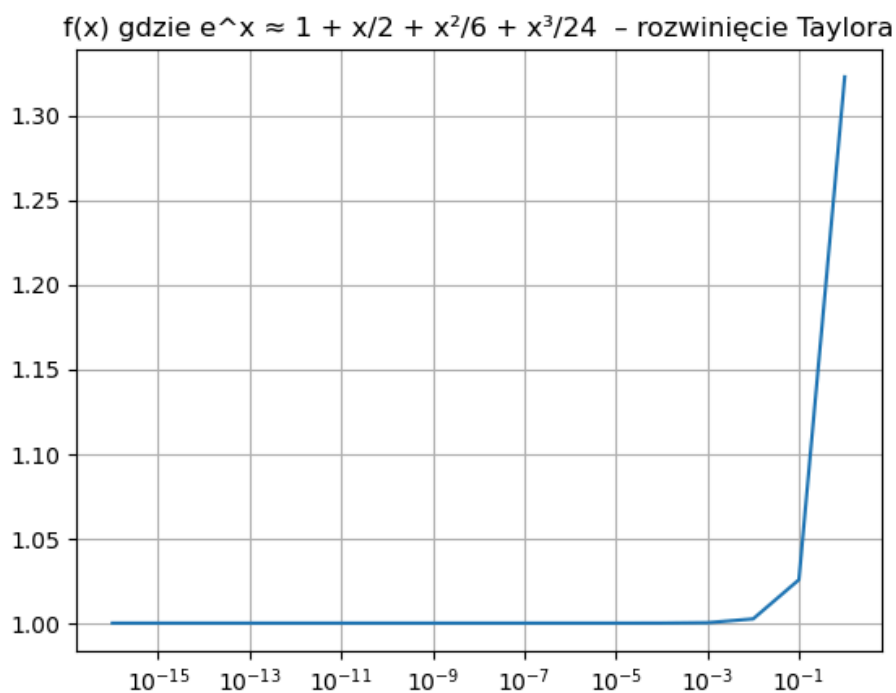
Aby uniknąć utraty precyzji dla wartości x bliskich zera (gdzie występuje zjawisko błędu kancellacji przy odejmowaniu $e^x - 1$), możemy wykorzystać rozwinięcie funkcji e^x w szereg Maclaurina (szereg Taylora wokół zera):

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots = 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24} + \dots$$

Podstawiając to rozwinięcie do wzoru na $f(x)$, problematyczna jedynka się redukuje, a my możemy podzielić całość przez x :

$$f(x) = \frac{\left(1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24} + \dots\right) - 1}{x} = 1 + \frac{x}{2} + \frac{x^2}{6} + \frac{x^3}{24} + \dots$$

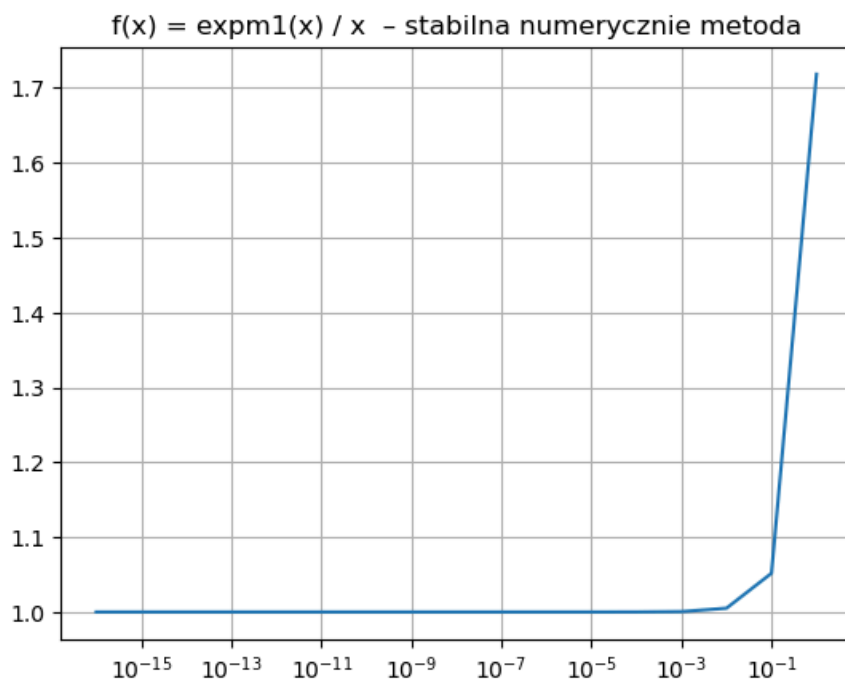
Obliczenie wartości $f(x)$ na podstawie uciętego szeregu (np. do wyrazu z x^3) przekształca problem odejmowania bliskich sobie liczb w proste dodawanie. Daje to doskonałe przybliżenie analityczne w otoczeniu zera i całkowicie eliminuje błąd numeryczny.



Rysunek 5: Wykres funkcji wyznaczonej za pomocą aproksymacji szeregiem Taylora.

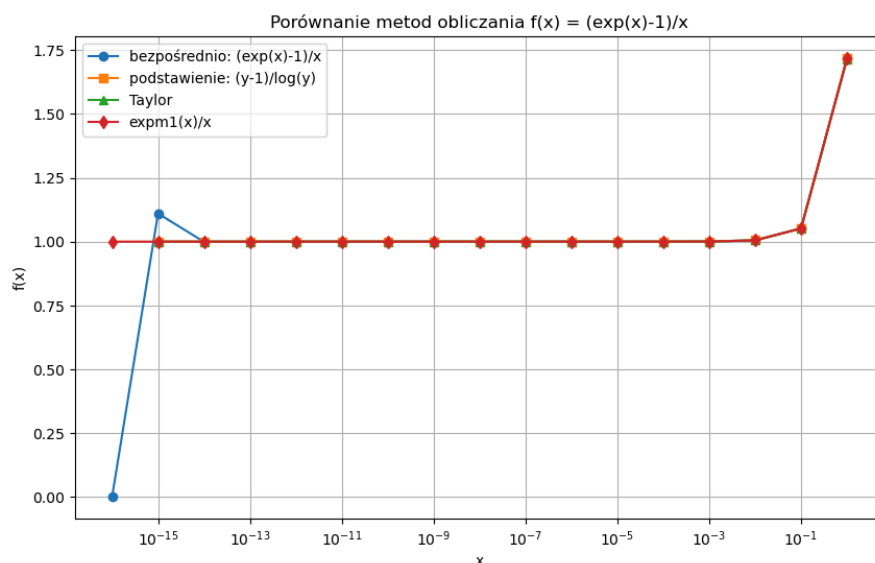
5. Zastosowanie funkcji expm1

W wielu językach programowania i bibliotekach matematycznych (np. `math.expm1` w Pythonie) zaimplementowana jest funkcja specjalnie zoptymalizowana do obliczania wyrażenia $e^x - 1$ dla małych x , omijająca błąd utraty precyzji.



Rysunek 6: Wykres funkcji obliczonej z wykorzystaniem wbudowanej funkcji $\text{expm1}(x)/x$.

6. Porównanie metod



Rysunek 7: Wykres porównujący działanie wszystkich metod obliczania funkcji

Skomentowanie otrzymanych wyników

Wszystkie metody prowadzą do wartości zbliżającej się do 1. W przypadku bezpośredniego obliczania mamy do czynienia ze spadkiem wartości do zera dla bardzo małych wartości x . Metoda Taylora jest mniej dokładna dla większych wartości x , ponieważ jest tylko przybliżeniem rozwinięcia funkcji. W metodzie z podstawieniem $y = \exp(x)$ oraz przy wykorzystaniu rozwinięcia Taylora pojawił się warning związany z dzieleniem przez bardzo małą wartość, co wynika z ograniczonej dokładności obliczeń zmiennoprzecinkowych. Warning nie wpływa jednak na końcowe wyniki, ponieważ wartości w tych punktach zostały poprawione. Najbardziej stabilne numerycznie wyniki daje metoda wykorzystująca funkcję `expm1`.

Zadanie 4

Zliczanie sumy n liczb zmiennoprzecinkowych pojedynczej precyzji z zakresu $[0, 1]$ na różne sposoby, porównanie błędów wszystkich metod. Sposoby sumowania:

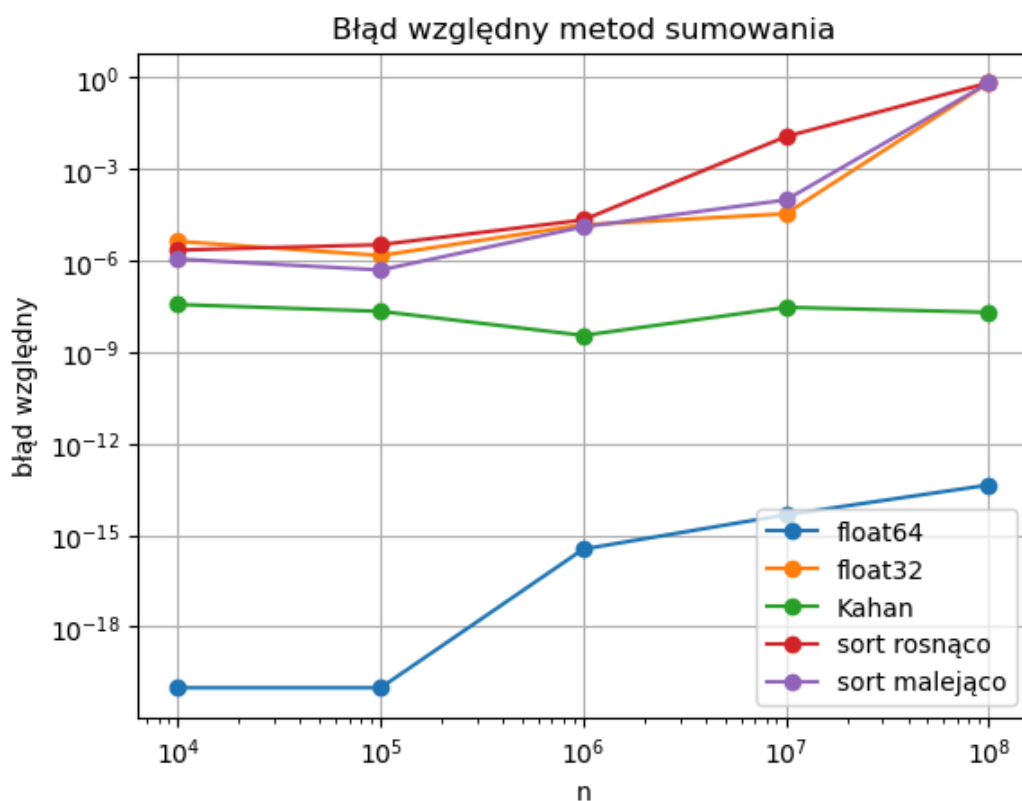
a - sumowanie w kolejności losowania, akumulator podwójnej precyzji

b - sumowanie w kolejności losowania, akumulator pojedynczej precyzji

c - sumowanie algorytmem Kahana z kompensacją, w kolejności losowania, akumulator pojedynczej precyzji

d - sumowanie w porządku rosnącym, akumulator pojedynczej precyzji

e - sumowanie w porządku malejącym, akumulator pojedynczej precyzji



Rysunek 8: Wykres błędów poszczególnych sposobów sumowań w zależności od n , za wynik dokładny przyjmujemy `math.fsum()`

Na wykresie przedstawiono zależność błędu względnego metod sumowania od liczby elementów n . Najmniejszy błąd uzyskano przy sumowaniu z akumulatorem podwójnej precyzji (`float64`), którego dokładność pozostaje bardzo wysoka dla wszystkich rozmiarów danych. Bardzo dobre wyniki daje również algorytm Kahana, którego błąd pozostaje niewielki i prawie stały. W przypadku sumowania w pojedynczej precyzji (`float32`) oraz sumowania posortowanych danych błąd rośnie wraz ze wzrostem liczby elementów, osiągając duże wartości dla największych zbiorów danych.

Bibliografia

- https://en.wikipedia.org/wiki/Machine_epsilon - artykuł na wikipedii na temat epsilon maszynowego wraz z tabelką z jego wartościami
- Wprowadzenie do laboratorium na platformie Teams w katalogu lab01